

Table of contents

Série 4 : Faible Précondition avec Boucles et Vérification	1
Méthodes	1
Weakest Precondition (WP) Calculus	1
Trouver et Prouver des Invariants de Boucle	2
Prouver la Terminaison avec des Variants	2
Vérification Formelle avec Why3	2
Rappel	2
Exemples Illustratifs	3
Exemple 1 : Calcul de WP et Invariant pour <code>stupidadd</code>	3
Exemple 2 : Variant de Terminaison pour <code>eucliddiv</code>	4
Synthèse	4

Série 4 : Faible Précondition avec Boucles et Vérification

Méthodes

Weakest Precondition (WP) Calculus

Méthode de raisonnement à rebours pour vérifier la correction partielle d'un programme. On calcule la condition la plus faible sur l'état initial qui garantit qu'après l'exécution du programme, la postcondition est satisfaite.

- **WP pour l'affectation** : $WP(x := e, Q) = Q[x \leftarrow e]$
- **WP pour la séquence** : $WP(S1; S2, Q) = WP(S1, WP(S2, Q))$
- **WP pour la boucle `while`** : C'est le point le plus complexe. Pour une boucle `while B do S done` avec un invariant I , on a :

$$WP(\text{while } B \text{ do } S \text{ done}, Q) = I \wedge \forall \vec{v}, ((B \wedge I) \Rightarrow WP(S, I)) \wedge ((\neg B \wedge I) \Rightarrow Q)$$

où $\vec{v}$ sont les variables modifiées dans la boucle. En pratique, la preuve se décompose en trois obligations :

1. **Initialisation** : La précondition implique l'invariant.
2. **Préservation** : Si l'invariant et la condition de boucle sont vrais avant le corps, alors l'invariant reste vrai après.
3. **Postcondition** : Si l'invariant est vrai et la condition de boucle est fausse, alors la postcondition est vraie.

Trouver et Prouver des Invariants de Boucle

Un invariant I est une propriété qui est vraie avant la première itération, préservée par chaque itération, et donc vraie à la sortie de la boucle.

- **Stratégie** : L'invariant relie souvent les variables mutables aux variables initiales (notées a_0, b_0) ou exprime une relation qui doit être maintenue (ex : $a + b = \text{constante}$).
- **Vérification** : On prouve la condition de préservation sous forme d'un triplet de Hoare : $\{B \wedge I\}S\{I\}$.

Prouver la Terminaison avec des Variants

Une variante V est une expression à valeurs entières (ou dans un ensemble bien-fondé) qui :

1. **Est minorée** : Typiquement $V \geq 0$ lorsque la condition de boucle B est vraie.
2. **Décroît strictement** : À chaque itération, sa valeur diminue.

Cela garantit que le nombre d'itérations est fini.

- **Stratégie** : Choisir une expression qui mesure clairement les “pas” restants (ex : une variable de décrémentation).

Vérification Formelle avec Why3

Why3 est un outil de vérification déductive de programmes.

- **Encodage** : Traduire le code, les spécifications (pré/postconditions), les invariants et les variants dans le langage de Why3.
- **Génération d'obligations de preuve** : Why3 génère automatiquement les formules logiques correspondant aux conditions de correction partielle et de terminaison.
- **Preuve** : Ces obligations peuvent être déchargées automatiquement par des solveurs (SMT) ou manuellement par des prouveurs interactifs.

Rappel

- **Logique de Hoare** : $\{P\}C\{Q\}$ signifie : si P est vrai avant l'exécution de C , et si C termine, alors Q est vrai après.
- **Règle de la boucle while** :

$$\frac{\{I \wedge B\}S\{I\}}{\{I\} \text{ while } B \text{ do } S \text{ done } \{\neg B \wedge I\}}$$

- **Terminaison** : Un variant V assure la terminaison si $\{I \wedge B \wedge V = K\}S\{V < K\}$ pour un K (et $V \geq 0$ sous $I \wedge B$).
- **Loi de l'affectation** : Pour prouver $\{P\}x := e\{Q\}$, il suffit que $P \Rightarrow Q[x \leftarrow e]$.

Exemples Illustratifs

Exemple 1 : Calcul de WP et Invariant pour stupidadd

Contexte :

Fonction `stupidadd(a, b)` qui ajoute `b` à `a` par incrémentations et décrémentations.

`stupidadd(a,b) = while (b > 0) do a := a + 1; b := b - 1 done; return a`

Spécification : $\{b \geq 0\}$ stupidadd $\{res = a_0 + b_0\}$.

Application :

1. **Choix de l'invariant I1** : $I_1 \equiv (b \geq 0) \wedge (a + b = a_0 + b_0)$.

- Il exprime que la somme totale reste constante et que `b` n'est jamais négatif pendant la boucle.

2. **Preuve de l'invariant** :

- **Initialisation** : Avant la boucle, `a = a0`, `b = b0`. Donc `b0 ≥ 0` et `b = 0` et `a + b = a0 + b0`. La précondition garantit `b0 ≥ 0`.
- **Préservation** : On doit montrer $\{b > 0 \wedge I_1\}a := a + 1; b := b - 1\{I_1\}$. Calculons la WP pour le corps de la boucle par rapport à I_1 :

$$WP(a := a + 1; b := b - 1, I_1) = WP(a := a + 1, WP(b := b - 1, I_1))$$

$$WP(b := b - 1, (b \geq 0) \wedge (a + b = a_0 + b_0)) = ((b - 1) \geq 0) \wedge (a + (b - 1) = a_0 + b_0)$$

$$WP(a := a + 1, (b - 1 \geq 0) \wedge (a + b - 1 = a_0 + b_0)) = (b - 1 \geq 0) \wedge ((a + 1) + b - 1 = a_0 + b_0)$$

Ce qui se simplifie en : $(b \geq 1) \wedge (a + b = a_0 + b_0)$.

La précondition du corps est $b > 0 \wedge I_1 \equiv (b > 0) \wedge (b \geq 0) \wedge (a + b = a_0 + b_0)$.

Il est évident que $(b > 0) \wedge (a + b = a_0 + b_0) \Rightarrow (b \geq 1) \wedge (a + b = a_0 + b_0)$.

La condition de préservation est donc vérifiée.

3. **Postcondition** : En sortie de boucle, $\neg B \wedge I_1$, i.e., $b \leq 0 \wedge b \geq 0 \wedge a + b = a_0 + b_0$. Cela implique $b = 0$ et donc $a = a_0 + b_0$. La fonction retourne `a`, donc $res = a_0 + b_0$.

Exemple 2 : Variant de Terminaison pour eucliddiv

Contexte :

Fonction calculant le quotient de la division euclidienne de a par b .

```
eucliddiv(a,b) = quot := 0; rest := a; while (rest >= b) do rest := rest - b; quot := quot + 1
```

Spécification : $\{a \geq 0 \wedge b > 0\}$ eucliddiv $\{a = b * quot + rest \wedge 0 \leq rest < b\}$. L'invariant est donné : $I_2 \equiv (rest \geq 0) \wedge (a = b * quot + rest)$.

Application :

Pour prouver la terminaison de la boucle **while** ($rest \geq b$) **do** ..., on définit un **variant**.

- **Choix du variant** : $V = rest$.
 - Sous l'invariant I_2 et la condition de boucle $rest \geq b$, on a $rest \geq 0$.
- **Preuve de décroissance** : On doit montrer qu'après une itération, **rest** a strictement diminué. Le corps de la boucle fait $rest := rest - b$. Sous la condition $rest \geq b > 0$, on a $rest - b < rest$. Donc, la valeur du variant V diminue strictement à chaque itération.
- **Conclusion sur la terminaison** : Puisque **rest** est un entier naturel qui décroît strictement à chaque itération et est minoré (par 0 d'après l'invariant), la boucle ne peut pas s'exécuter indéfiniment. Le programme termine.

Synthèse

Cette série met en lumière les éléments centraux de la vérification de programmes impératifs avec boucles :

1. Le **calcul de la plus faible précondition** fournit un cadre systématique pour la correction partielle.
2. La découverte d'un **invariant de boucle pertinent** est l'étape créative clé, capturant l'essence de l'algorithme.
3. La preuve de **terminaison** s'appuie sur la construction d'un **variant**, une mesure entière strictement décroissante et minorée.
4. L'outil **Why3** permet d'automatiser la génération et la décharge des preuves correspondant à ces raisonnements.

La maîtrise de ces concepts est fondamentale pour spécifier et prouver rigoureusement la correction de programmes séquentiels.